Dr. Vishwanath Karad
MIT WORLD PEACE UNIVERSITY | PUNE
TECHNOLOGY, RESEARCH, SOCIAL INNOVATION & PARTNERSHIPS

Foundations of Programming

Assignment No. : 10

PROBLEM STATEMENT:

Write a program in C to simulate a simple calculator that performs basic arithmetic operations such as addition, subtraction, multiplication, and division. The calculator should also perform special operations like computing x^y (power) and x! (factorial).

OBJECTIVE:

• To understand the use of switch-case statements in C.• To perform basic arithmetic and special mathematical operations.• To enhance problem-solving skills using menu-driven programs.

THEORY:

A calculator is a basic computing device used to perform arithmetic operations. In C programming, a simple calculator can be implemented using conditional or switch-case statements. Arithmetic operations like addition, subtraction, multiplication, and division are fundamental operations. Special operations such as power (x^y) and factorial (x!) require loops for repeated calculations.

PLATFORM:Linux

ALGORITHM:

1. Start
2. Display menu:
    1. Addition
    2. Subtraction
    3. Multiplication
    4. Division
    5. Power (x^y)
    6. Factorial (x!)
    7. Exit
3. Take user choice
4. If choice is 1–4 or 5:
- Input two numbers (x and y)
5. If choice is 6 (Factorial):
- Input one number (x)
6. Perform operation based on choice:
- If 1 → result = x + y
- If 2 → result = x - y
- If 3 → result = x × y
- If 4 →
    - If y = 0 → print “Division not possible”
    - Else → result = x ÷ y
- If 5 → result = x^y
- If 6 →
    - Set fact = 1
    - Repeat from 1 to x:
      → fact = fact × i
7. Display result
8. Repeat steps until user selects Exit
9. Stop

C PROGRAM:

```
#include <stdio.h>

#include <math.h>

int main() {

   float x, y, result;

   int choice, i;

   int fact = 1;

   printf("Simple Calculator\n");

   printf("1. Addition\n");

   printf("2. Subtraction\n");

   printf("3. Multiplication\n");

   printf("4. Division\n");

   printf("5. Power (x^y)\n");

   printf("6. Factorial (x!)\n");

   printf("Enter your choice: ");

   scanf("%d", &choice);

   if(choice >= 1 && choice <= 5) {

      printf("Enter two numbers: ");

      scanf("%f %f", &x, &y);

   }

   else if(choice == 6) {

      printf("Enter a number: ");

      scanf("%f", &x);

   }

switch(choice) {

         case 1:

                  result = x + y;

                  printf("Result = %.2f", result);

                  break;

         case 2:
```

```
        result = x - y;
        printf("Result = %.2f", result);
        break;
case 3:
        result = x * y;
        printf("Result = %.2f", result);
        break;
case 4:
        if (y == 0)
        printf("N.D.");
        else {
        result = x / y;
        printf("Result = %.2f", result);
                }
        break;
case 5:
        result = pow(x, y);
        printf("Result = %.2f", result);
        break;
case 6:
        if(x < 0)
        printf("Factorial not defined for negative numbers");
        else {
         for(i = 1; i <= (int)x; i++)
         fact = fact * i;
        printf("Factorial = %d", fact);
        }
        break;
default:
        printf("Invalid choice");
```

}

return 0;

}

```
#include <stdio.h>
#include <math.h>
int main() {
    float x, y, result;
    int choice, i;
    int fact = 1;

    printf("Simple Calculator\n");
    printf("1. Addition\n");
    printf("2. Subtraction\n");
    printf("3. Multiplication\n");
    printf("4. Division\n");
    printf("5. Power (x^y)\n");
    printf("6. Factorial (x!)\n");

    printf("Enter your choice: ");
    scanf("%d", &choice);
    if(choice >= 1 && choice <= 5) {
        printf("Enter two numbers: ");
        scanf("%f %f", &x, &y);
    }
    else if(choice == 6) {
        printf("Enter a number: ");
        scanf("%f", &x);
    }
switch(choice) {
        case 1:
                         result = x + y;
                        printf("Result = %.2f", result);
                        break;
        case 2:
                        result = x - y;
                        printf("Result = %.2f", result);
                        break;

        case 3:
                        result = x * y;
                        printf("Result = %.2f", result);
                        break;
        case 4:
                        if (y == 0)
                                printf("N.D.");
                        else {
                                result = x / y;
                                printf("Result = %.2f", result);
                                }
                                break;
        case 5:
                        result = pow(x, y);
                        printf("Result = %.2f", result);
                        break;

                case 6:
                        if(x < 0)
                                printf("Factorial not defined for negative numbers");
                        else {
                         for(i = 1; i <= (int)x; i++)
                                 fact = fact * i;
                        printf("Factorial = %d", fact);
                                }
                                break;

                default:
                        printf("Invalid choice");
    }

    return 0;
}
```

Example Input:Choice: 1Numbers: 10 20

OUTPUT:

Calculator Menu:

1. Addition

2. Subtraction

3. Multiplication

4. Division

5. Power (x^y)

6. Factorial (x!)

Enter your choice: 1

Enter two numbers: 10 5

Result = 15.00

CONCLUSION:

Thus, a simple calculator program was successfully implemented using C programming. The program demonstrates the use of switch-case statements, loops, and arithmetic operations.

FAQs:

1. What is a simple calculator program?
2. What is a simple calculator program is an important programming concept because it supports clear problem solving, structured logic, and correct implementation. Understanding it helps in writing programs that are easier to develop, test, and maintain.
3. Why is switch–case preferred in calculator programs?
4. The switch-case statement is a multi-way decision control structure used when one expression must be compared against several constant choices. It makes menu-based programs easier to read and maintain than writing many separate if-else conditions.
5. How does the program handle division by zero?
6. How does the program handle division by zero is an important programming concept because it supports clear problem solving, structured logic, and correct implementation. Understanding it helps in writing programs that are easier to develop, test, and maintain.
7. How is the power operation (x^y) implemented without using math library functions?
8. A function is a reusable block of code designed to perform a specific task. Functions improve modularity, reduce repetition, and make a program easier to read, test, and maintain.
9. What is factorial and how is it calculated using loops in C?
10. What is factorial and how is it calculated using loops in C is an important programming concept because it supports clear problem solving, structured logic, and correct implementation. Understanding it helps in writing programs that are easier to develop, test, and maintain.